`PrivateAssets="All"` to the relevant package reference within the project file:

```
<PackageReference Include="Microsoft.EntityFrameworkCore.Tools"
➥ Version="6.0.0" PrivateAssets="All" />
```

ENABLE STDOUT LOGGING

If you haven't already incorporated logging within the application, enabling basic logging provides an easy way to get full details of startup errors within the deployed application. In listing 14.27, you can see within the `aspNetCore` section that `stdout-LogEnabled` is set to `false` by default. Set this to `true`, and text-based log files will be generated to a folder named logs in the root of the application. The folder will be created if one doesn't exist already:

```
<aspNetCore processPath=".\WebApplication1.exe" stdoutLogEnabled="true"
➥ stdoutLogFile=".\logs\stdout" hostingModel="InProcess" />
```

You should disable `stdout` logging once any errors have been resolved. It should not be used for routine logging; you should instead use a third-party library that incorporates log file management.

FIXING THE SDK VERSION USED FOR PUBLISHING

The `<TargetFramework>` node in your application's *.csproj* file specifies the version of the .NET Framework that your application targets. However, this does not prevent your application being built or published against a newer version of the framework. The `dotnet` commands